

Raport z testów oprogramowania

1. Znalezione błędy podczas użytkowania aplikacji i sposoby w jaki sobie z nimi poradziliśmy

ID	Opis wykrytego błędu	Opis rozwiązania	Status
1	Wygenerowany kod QR dla pracownika nie zwracał hashu QR, który był generowany razem z utworzeniem użytkownika (zwracał ID użytkownika w bazie) co uniemożliwiało uwierzytelnienie pracownika poprzez kod QR	W <i>/Pages/AdminPage.tsx</i> zmieniono atrybut przekazywany do komponentu generującego grafikę QR (QRCodeCanvas) – zamiast <i>user.id</i> przekazywany jest teraz <i>user.qr_token</i> , który jest wymagany przez <i>AccessService</i> do weryfikacji uprawnień.	Poprawione
2	System akceptował zdjęcie wyświetlone na ekranie smartfona podstawione przed kamerę jako żywą osobę. Brak weryfikacji głębi lub kontekstu sceny.	W <i>app/core/biometrics.py</i> metoda <i>detect_liveness</i> oblicza stosunek pola twarzy do całej klatki. Jeśli <i>face_area / total_area < 0.02</i> , system odrzuca obraz jako próbę spoofingu (zbyt mała/płaska twarz).	Poprawione
3	Użytkownik w ruchu generuje rozmazany obraz. Bez filtra system próbuje dopasować embedding do szumu, co powoduje, że użytkownik długo czeka na weryfikację zdjęcia bez komunikatu zwrotnego	W <i>biometrics.py</i> funkcja <i>check_image_quality</i> używa wariacji laplasianu do zmierzenia ostrości zdjęcia. Jeśli wynik jest poniżej <i>BLUR_THRESHOLD</i> , zdjęcie jest odrzucane przed ciężkim procesowaniem AI.	Poprawione
4	Słabe wykrywanie twarzy przy słabym oświetleniu	W <i>services/image_facade_service.py</i> zaimplementowano algorytmy CLAHE	Poprawione

		(wyrównanie histogramu) oraz korekcję Gamma jako mechanizm naprawczy, który automatycznie przetwarza obraz	
5	Przy tworzeniu nowego użytkownika i dodawaniu go do bazy danych, tworzony jest jednocześnie hash QR. Nie możemy dezaktywować kodu i generować nowego bez usuwania pracownika	W <i>services/user_services.py</i> dodano funkcję, która umożliwia wygenerowanie nowego QR dla istniejącego pracownika w bazie	Poprawione
6	Generowanie nowego kodu QR dla pracownika nie zmieniało okresu ważności QR. Po wygenerowaniu nowego QR i tak byłby nie ważny, bo data ważności byłaby przedawniona	W <i>services/user_services.py</i> dodano funkcjonalność do aktualizowania daty w momencie regenerowania QR w funkcji <i>regenerate_qr_token()</i>	Poprawione
7	Przy aktualizacji danych użytkownika wyskakiwał błąd w postaci okienka pop-up i nie można było aktualizować danych (błąd backendu <code>AttributeError: 'UserUpdateRequest' object has no attribute 'get'</code>)	W <i>backend/app/api/v1/admin.py</i> dodano konwersję listy obiektów Pydantic na listę słowników (metoda <i>.model_dump()</i>) przed przekazaniem ich do serwisu obsługującego aktualizację (<i>user_service.update_users</i>), który oczekiwał formatu słownikowego.	Poprawione
8	Przy próbie dodania użytkownika API zwracało błąd 500 Internal Server Error, ponieważ warstwa serwisu próbowała odczytać dane metodą <i>.get()</i> , która nie jest dostępna dla obiektów Pydantic przekazywanych	W <i>backend/app/api/v1/admin.py</i> dodano konwersję listy obiektów użytkowników na listę słowników przy użyciu metody <i>.model_dump()</i> przed przekazaniem ich do funkcji <i>create_users_bulk</i> w warstwie serwisu	Poprawione

	bezpośrednio z kontrolera		
9	W zakładce do edycji danych pracownika przekazywany format daty (od zmiany daty ważności QR) był niezgodny z formatem ważności daty w bazie danych	W <i>backend/app/schemas/admin.py</i> dodano brakujące pole <i>qr_valid_until</i> do modelu walidacji <i>UserUpdateRequest</i> , a w <i>frontend/src/Pages/AdminPage.tsx</i> zaimplementowano formatowanie daty do pełnego standardu (dodanie sekund), aby backend mógł poprawnie sparsować czas.	Poprawione
10	Generowane raporty PDF nie wyświetlały poprawnie polskich znaków diakrytycznych (np. ą, ę, ś), zastępując je losowymi symbolami	W <i>frontend/src/Pages/AdminPage.tsx</i> zaimplementowano funkcję <i>applyPolishFont</i> , która przed wygenerowaniem raportu dynamicznie pobiera i łąduje do dokumentu czcionkę Roboto (zakodowaną w Base64 lub z URL), która posiada pełne wsparcie dla polskich glifów	Poprawione
11	Kamera mogła przesyłać zdjęcia tła, bez osoby, co powodowało że użytkownik czekał aż zostanie wykryta twarz, pozostając na ekranie weryfikacji przez długi czas	W <i>backend/app/services/image_facade_service.py</i> (oraz <i>access_service.py</i>) dodano wstępną walidację obecności twarzy. Jeśli algorytm detekcji nie znajdzie twarzy na zdjęciu, metoda natychmiast zwraca status <i>NO_FACE</i> i pomija etap przeszukiwania bazy danych, oszczędzając czas procesora.	Poprawione

12	Podczas przeglądania logów z uwierzytelniania użytkowników nie były widoczne komunikaty z niepoprawnym kodem QR	W backend/app/db/repositorie s/log_repo.py zmodyfikowano zapytanie SQL w metodzie get_logs_since, dodając pole qr_content do listy pobieranych kolumn, co pozwoliło na przesłanie treści błędnego kodu do frontendu. Potem w backend/app/db/repositorie s/log_repo.py zmodyfikowano zapytanie SQL w metodzie get_logs_since, dodając pole qr_content do listy pobieranych kolumn, co pozwoliło na przesłanie treści błędnego kodu do frontendu	Poprawione
13	Po zeskanowaniu nieprawidłowego kodu QR (takiego, którego hash nie istniał w bazie), system przechodził do skanowania twarzy a następnie dopiero pokazywał komunikat o nierozpoznanym kodzie QR	W <i>UserPage.tsx</i> wdrożono dwuetapową weryfikację. Zamiast wysyłać kod QR razem ze zdjęciem twarzy w jednym żądaniu, system najpierw wywołuje nową funkcję <i>verifyQrInDatabase</i>, która sprawdza ważność samego tokena (<i>api.checkQr</i>). Jeśli kod jest niepoprawny, maszyna stanów przechodzi bezpośrednio do DENIED z pominięciem etapu skanowania twarzy, co oszczędza czas użytkownika i zasoby serwera.	Poprawione

14	Po zeskanowaniu kodu QR, który utracił już swoją ważność wyskakiwał komunikat o błędzie połączenia z serwerem	W <code>services/access_services.py</code> dodano warunek w sprawdzaniu daty ważności sprawdzanego tokenu oraz zmodyfikowano tabelę w bazie danych, aby przechowywała wartość tokena "EXPIRED_QR"	Poprawione
----	---	---	------------

2. Przetestowana funkcjonalność

ID	Funkcjonalność	Uwagi
1	Dodawanie nowych pracowników	Wszystko działa poprawnie, czasami jeśli zdjęcie ma za duży rozmiar, długo jest przetwarzane i wyliczany embedding z twarzy
2	Aktualizowanie danych użytkowników	Zmiana danych działa w pełni poprawnie, ten sam problem ze zdjęciem co z dodawaniem nowych użytkowników - jeśli zdjęcie ma duży rozmiar, dłużej są wyliczane embeddingi z twarzy użytkownika
3	Generowanie raportów z logowania użytkowników w formacie pdf	Działa poprawnie, brak uwag
4	Skanowanie kodu QR, którego nie ma w bazie	Działa prawidłowo, od razu wyskakuje komunikat o tym, że jest próba skanowania niepoprawnego kodu QR
5	Skanowanie kodu QR w słabych warunkach oświetleniowych	Czasami trzeba poczekać chwilę, aby kamera wyłapała ostrość na ekran z kodem QR. Czas oczekiwania na zeskanowanie QR zależy od jakości używanej kamery
6	Weryfikacja tożsamości użytkownika poprzez wyświetlanie	Zdarzały się przypadki, w których system przepuszczał takie

	zdjęcia na telefonie	zdjęcia, ale w zostawały one też odrzucane. W bazie znajdują się zdjęcia z witryny https://thispersondoesnotexist.com , więc to był jedyny możliwy sposób sprawdzenia czy system je przepuści
7	Skanowanie kodu QR, który już stracił ważność	Działa poprawnie, system wyrzuca komunikat o przedawnieniu kodu QR
8	Sprawdzenie jak działa automatyczne usuwanie logów starszych niż dzień (w wymaganiach było 6 miesięcy, ale fizycznie nie dało się tego zrobić)	Działa poprawnie, komunikaty wyświetlane są w logach konsolowych
9	Sprawdzaliśmy jak system zachowa się w sytuacji, gdy kod QR będzie poprawny, a dane twarzy nie będą w bazie	System poprawnie reagował, nie przepuszcza dalej i w logach widać zdjęcie intruza
10	Sprawdziliśmy jak zachowa się system w sytuacji gdy przez bramki będzie chciało przejść 50 osób na raz	Wynik testu był pozytywny, celem było sprawdzenie czy przetworzy dużo zapytań bez generowania błędów i w odpowiednim czasie
11	Jak system poradzi sobie z awarią (błędem połączenia z bazą), która ma miejsce po weryfikacji biometrycznej, ale przed zapisaniem logu	Działa poprawnie, zwraca błąd <i>500 Internal Server Error</i> i nie przyznaje dostępu, zamiast pozostawiać system w niepewnym stanie
12	Jak API <code>/api/v1/access/verify</code> reaguje na błędne zapytania (gdy nie ma wymaganych pól)	Działa tak jak oczekujemy, wyrzuca błąd <i>422 Unprocesable Entity</i>